INTRODUCTION AUX BASES DE DONNÉES · L2

Du modèle au SQL

Corrigé du TD4 : modèles, tables et requêtes des trois exercices

```sql
SELECT * FROM corrige WHERE td = 4;
```

MODE D’EMPLOI

Avant de comparer avec votre copie

Plusieurs écritures

Alias, ordre des jointures, JOIN … ON ou FROM … WHERE, NOT IN ou LEFT JOIN : seul le résultat compte.

Des types au choix

Ils se déduisent des valeurs du sujet. Une autre longueur de VARCHAR, ou NUMERIC à la place d’INTEGER, conviennent aussi.

Pourquoi DISTINCT

Un nom n’est pas une clé : deux lignes peuvent porter le même. Quand la question demande une liste de noms, DISTINCT évite les répétitions.

Chaque exercice suit le même plan : les modèles, la création des tables, puis les onze questions dans l’ordre du sujet.

EXERCICE 1

Gestion de la production agro-industrielle

Des usines livrent des produits à des magasins, par lots.

EXERCICE 1 · MODÈLES

Le modèle logique

```
magasin(ref_mag, nom_mag, ville)
produit(ref_prod, nom_prod, version, poids)
usine(ref_usine, nom_usine, ville)
lot(#ref_prod, #ref_usine, #ref_mag, quantite)
```

Souligné : clé primaire · # : clé étrangère

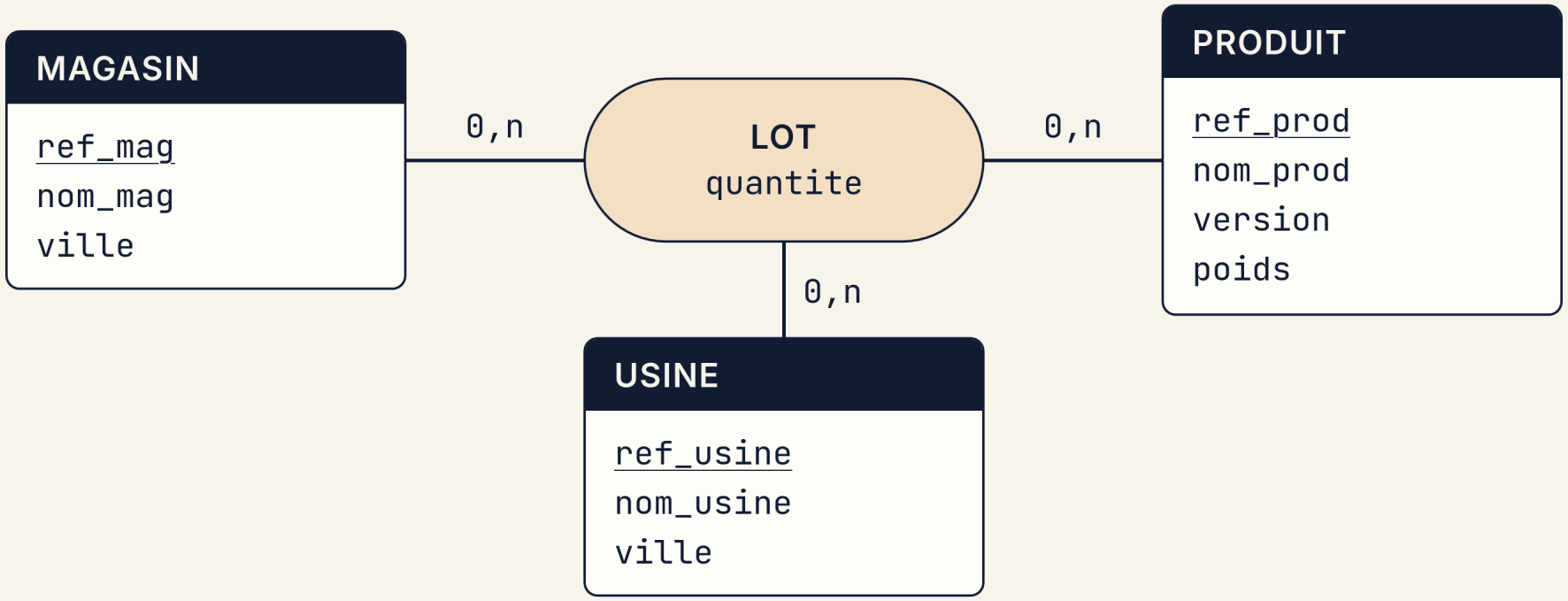
Chaque INSERT du sujet donne une table et la liste de ses colonnes.

lot ne contient que les références des trois autres tables, plus quantite : c'est une table d'association.

Sa clé primaire réunit les trois références, qui sont aussi trois clés étrangères.

EXERCICE 1 · MODÈLES

Le modèle conceptuel



Association ternaire : un lot relie un produit, une usine et un magasin, et porte la quantité.

Cardinalités 0,n : le sujet n'impose aucun minimum, et la question 2 suppose des magasins sans lot.

EXERCICE 1 · CRÉATION DES TABLES

Les tables sans clé étrangère

```sql
CREATE TABLE magasin (
    ref_mag  INTEGER PRIMARY KEY,
    nom_mag  VARCHAR(50),
    ville    VARCHAR(50)
);
```

```sql
CREATE TABLE usine (
    ref_usine  INTEGER PRIMARY KEY,
    nom_usine  VARCHAR(50),
    ville      VARCHAR(50)
);
```

```sql
CREATE TABLE produit (
    ref_prod  INTEGER PRIMARY KEY,
    nom_prod  VARCHAR(50),
    version   VARCHAR(30),
    poids     INTEGER
);
```

INTEGER pour les références et pour le poids, qui vaut 5 dans le sujet.

VARCHAR pour les textes. Les longueurs 50 et 30 sont un choix : d'autres conviennent.

EXERCICE 1 · CRÉATION DES TABLES

La table d’association

```
CREATE TABLE lot (
    ref_prod   INTEGER,
    ref_usine  INTEGER,
    ref_mag    INTEGER,
    quantite   INTEGER,
    PRIMARY KEY (ref_prod, ref_usine, ref_mag),
    FOREIGN KEY (ref_prod)  REFERENCES produit(ref_prod),
    FOREIGN KEY (ref_usine) REFERENCES usine(ref_usine),
    FOREIGN KEY (ref_mag)   REFERENCES magasin(ref_mag)
);
```

Clé primaire : les trois références réunies, sur une seule ligne.

Clés étrangères : une par table reliée, vers sa clé primaire.

Attention : l’INSERT du sujet dans lot cite le magasin 14, absent de la table magasin. Il est refusé tant que ce magasin n’est pas inséré.

EXERCICE 1 · REQUÊTES

Questions 1 et 2

1 Le contenu de la table magasin

```sql
SELECT * FROM magasin;
```

2 Le nom des magasins n'ayant aucune quantité

AVEC NOT IN

```sql
SELECT nom_mag
FROM magasin
WHERE ref_mag NOT IN (
    SELECT ref_mag FROM lot
);
```

AVEC LEFT JOIN

```sql
SELECT m.nom_mag
FROM magasin m
LEFT JOIN lot l
       ON l.ref_mag = m.ref_mag
WHERE l.ref_mag IS NULL;
```

Aucune quantité : le magasin n'apparaît dans aucune ligne de lot. Les deux écritures donnent le même résultat.

EXERCICE 1 · REQUÊTES

Questions 3 à 5

3

Le nom des magasins, sans doublons

```
SELECT DISTINCT nom_mag
FROM magasin;
```

Deux magasins peuvent porter le même nom : DISTINCT ne le cite qu'une fois.

4

Le nom des villes dans lesquelles une usine est implantée

```
SELECT DISTINCT ville
FROM usine;
```

Une ville qui compte plusieurs usines n'est citée qu'une fois, grâce à DISTINCT.

5

Le nom et la version de chaque produit

```
SELECT nom_prod, version
FROM produit;
```

Deux colonnes, séparées par une virgule ; toutes les lignes sont gardées.

EXERCICE 1 · REQUÊTES

Question 6

6 Le nom de chaque produit livré au magasin « Stock10 »

```
SELECT DISTINCT p.nom_prod
FROM lot l
JOIN produit p ON p.ref_prod = l.ref_prod
JOIN magasin m ON m.ref_mag  = l.ref_mag
WHERE m.nom_mag = 'Stock10';
```

nom_prod est dans produit et la condition porte sur magasin : **lot** relie les deux.

DISTINCT : un produit livré par deux usines au même magasin ne sort qu'une fois.

La table usine ne sert pas ici : inutile de la joindre.

EXERCICE 1 · REQUÊTES

Question 7

7 Le poids de chaque lot

```
SELECT l.ref_prod, l.ref_usine, l.ref_mag,
       l.quantite * p.poids AS poids_lot
FROM lot l
JOIN produit p ON p.ref_prod = l.ref_prod;
```

Poids d'un lot = quantité × poids unitaire du produit.

quantite est dans lot, **poids** dans produit : la jointure est nécessaire.

Les trois références identifient le lot ; **AS** nomme la colonne calculée.

EXERCICE 1 · REQUÊTES

Question 8

8 Le nom des magasins approvisionnés en produits « Pizza4Fromages » par l'usine « ITFood »

```
SELECT DISTINCT m.nom_mag
FROM lot l
JOIN magasin m ON m.ref_mag   = l.ref_mag
JOIN produit p ON p.ref_prod  = l.ref_prod
JOIN usine u   ON u.ref_usine = l.ref_usine
WHERE p.nom_prod = 'Pizza4Fromages'
  AND u.nom_usine = 'ITFood';
```

Trois tables utiles autour de lot : magasin pour l'affichage, produit et usine pour les conditions.

Les deux conditions sont réunies par **AND**.

DISTINCT par prudence : deux versions du même produit donneraient deux fois le même magasin.

EXERCICE 1 · REQUÊTES

Question 9

9 La liste des lots de produits dont le nom contient « se »

```
SELECT l.*
FROM lot l
JOIN produit p ON p.ref_prod = l.ref_prod
WHERE p.nom_prod LIKE '%se%';
```

l.* désigne toutes les colonnes de lot, et elles seules.

Un **%** de chaque côté : « se » peut se trouver n'importe où dans le nom.

LIKE distingue les majuscules : « Sel » ne serait pas trouvé, sauf avec ILIKE.

EXERCICE 1 · REQUÊTES

Questions 10 et 11

10 La quantité de chaque produit par magasin

```
SELECT ref_prod, ref_mag,
       SUM(quantite) AS quantite_totale
FROM lot
GROUP BY ref_prod, ref_mag;
```

Un magasin peut recevoir le même produit de plusieurs usines : **SUM** additionne ces lots.

Pour afficher les noms, joindre produit et magasin et ajouter les noms au GROUP BY.

11 Les noms des produits qui pèsent moins que 10

```
SELECT nom_prod
FROM produit
WHERE poids < 10;
```

Un nombre s'écrit sans apostrophes. Le signe < exclut la valeur 10 elle-même.

EXERCICE 2

Gestion des réservations d’appareils

Des laboratoires réservent des appareils sur des créneaux horaires.

EXERCICE 2 · MODÈLES

Le modèle logique

```
laboratoire(ref_lab, nom_lab, batiment)
appareil(ref_app, nom_app, type, tarif_horaire)
creneau(ref_creneau, jour, heure_debut)
reservation(#ref_lab, #ref_app, #ref_creneau, nb_heures)
```

Souligné : clé primaire · # : clé étrangère

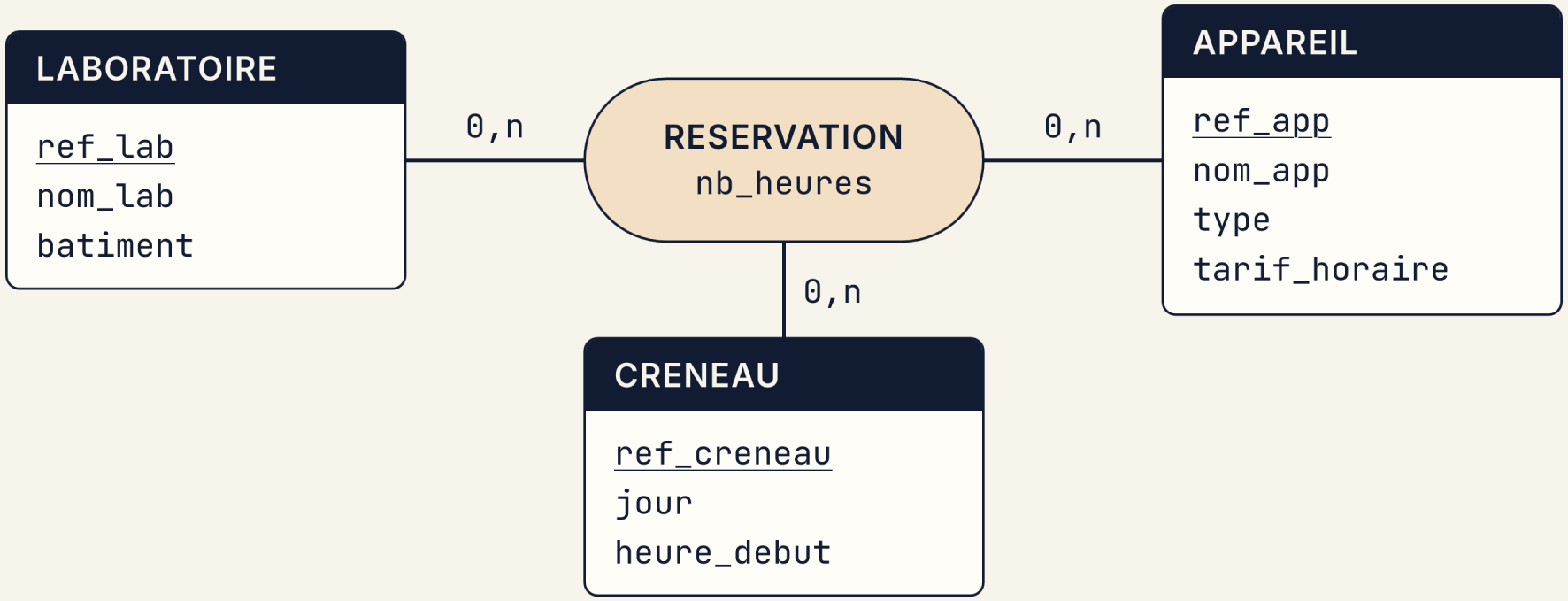
Chaque INSERT du sujet donne une table et la liste de ses colonnes.

reservation ne contient que les références des trois autres tables, plus nb_heures : c'est une table d'association.

Sa clé primaire réunit les trois références, qui sont aussi trois clés étrangères.

EXERCICE 2 · MODÈLES

Le modèle conceptuel



Association ternaire : une réservation relie un laboratoire, un appareil et un créneau, et porte le nombre d'heures.

Cardinalités 0,n : le sujet n'impose aucun minimum, et la question 2 suppose des laboratoires sans réservation.

EXERCICE 2 · CRÉATION DES TABLES

Les tables sans clé étrangère

```sql
CREATE TABLE laboratoire (
    ref_lab   INTEGER PRIMARY KEY,
    nom_lab   VARCHAR(50),
    batiment  VARCHAR(10)
);
```

```sql
CREATE TABLE creneau (
    ref_creneau  INTEGER PRIMARY KEY,
    jour         VARCHAR(10),
    heure_debut  TIME
);
```

```sql
CREATE TABLE appareil (
    ref_app         INTEGER PRIMARY KEY,
    nom_app         VARCHAR(50),
    type            VARCHAR(30),
    tarif_horaire   INTEGER
);
```

TIME pour heure_debut : la valeur 08:00 est une heure.

INTEGER pour tarif_horaire, qui vaut 20 dans le sujet ; NUMERIC(6,2) conviendrait pour des tarifs à décimales.

EXERCICE 2 · CRÉATION DES TABLES

La table d’association

```
CREATE TABLE reservation (
    ref_lab      INTEGER,
    ref_app      INTEGER,
    ref_creneau  INTEGER,
    nb_heures    INTEGER,
    PRIMARY KEY (ref_lab, ref_app, ref_creneau),
    FOREIGN KEY (ref_lab)     REFERENCES laboratoire(ref_lab),
    FOREIGN KEY (ref_app)     REFERENCES appareil(ref_app),
    FOREIGN KEY (ref_creneau) REFERENCES creneau(ref_creneau)
);
```

Clé primaire : les trois références réunies, sur une seule ligne.

Clés étrangères : une par table reliée, vers sa clé primaire.

Cette table se crée en dernier ; les INSERT suivent le même ordre.

EXERCICE 2 · REQUÊTES

Questions 1 et 2

1 Le contenu de la table laboratoire

```sql
SELECT * FROM laboratoire;
```

2 Le nom des laboratoires n'ayant aucune réservation

AVEC NOT IN

```sql
SELECT nom_lab
FROM laboratoire
WHERE ref_lab NOT IN (
    SELECT ref_lab FROM reservation
);
```

AVEC LEFT JOIN

```sql
SELECT l.nom_lab
FROM laboratoire l
LEFT JOIN reservation r
       ON r.ref_lab = l.ref_lab
WHERE r.ref_lab IS NULL;
```

Aucune réservation : le laboratoire n'apparaît dans aucune ligne de reservation. Les deux écritures donnent le même résultat.

EXERCICE 2 · REQUÊTES

Questions 3 à 5

3

Le nom des appareils, sans doublons

```
SELECT DISTINCT nom_app
FROM appareil;
```

Deux appareils de types différents peuvent porter le même nom : DISTINCT ne le cite qu'une fois.

4

Le nom des bâtiments dans lesquels un laboratoire est installé

```
SELECT DISTINCT batiment
FROM laboratoire;
```

Un bâtiment qui abrite plusieurs laboratoires n'est cité qu'une fois, grâce à DISTINCT.

5

Le nom et le type de chaque appareil

```
SELECT nom_app, type
FROM appareil;
```

Deux colonnes, séparées par une virgule ; toutes les lignes sont gardées.

EXERCICE 2 · REQUÊTES

Question 6

6 Le nom de chaque appareil réservé par le laboratoire « BioMol »

```
SELECT DISTINCT a.nom_app
FROM reservation r
JOIN appareil a    ON a.ref_app = r.ref_app
JOIN laboratoire l ON l.ref_lab = r.ref_lab
WHERE l.nom_lab = 'BioMol';
```

nom_app est dans appareil et la condition porte sur laboratoire : **reservation** relie les deux.

DISTINCT : un appareil réservé sur plusieurs créneaux ne sort qu'une fois.

La table creneau ne sert pas ici : inutile de la joindre.

EXERCICE 2 · REQUÊTES

Question 7

7 Le coût de chaque réservation, au tarif horaire de l'appareil

```
SELECT r.ref_lab, r.ref_app, r.ref_creneau,
       r.nb_heures * a.tarif_horaire AS cout
FROM reservation r
JOIN appareil a ON a.ref_app = r.ref_app;
```

Coût d'une réservation = nombre d'heures × tarif horaire de l'appareil.

nb_heures est dans reservation, **tarif_horaire** dans appareil : la jointure est nécessaire.

Les trois références identifient la réservation ; **AS** nomme la colonne calculée.

EXERCICE 2 · REQUÊTES

Question 8

8 Le nom des laboratoires ayant réservé l'appareil « Microscope » sur le créneau du lundi à 08:00

```
SELECT DISTINCT l.nom_lab
FROM reservation r
JOIN laboratoire l ON l.ref_lab     = r.ref_lab
JOIN appareil a    ON a.ref_app     = r.ref_app
JOIN creneau c     ON c.ref_creneau = r.ref_creneau
WHERE a.nom_app = 'Microscope'
  AND c.jour = 'lundi'
  AND c.heure_debut = '08:00';
```

Trois tables utiles autour de reservation : laboratoire pour l'affichage, appareil et creneau pour les conditions.

Le créneau se reconnaît à deux colonnes, **jour** et **heure_debut**, d'où trois conditions.

DISTINCT par prudence : deux microscopes de types différents donneraient deux fois le même laboratoire.

EXERCICE 2 · REQUÊTES

Question 9

9 La liste des réservations d'appareils dont le nom contient « scope »

```
SELECT r.*
FROM reservation r
JOIN appareil a ON a.ref_app = r.ref_app
WHERE a.nom_app LIKE '%scope%';
```

r.* désigne toutes les colonnes de reservation, et elles seules.

Un **%** de chaque côté : « scope » peut se trouver n'importe où dans le nom.

LIKE distingue les majuscules : « Scope » ne serait pas trouvé, sauf avec ILIKE.

EXERCICE 2 · REQUÊTES

Questions 10 et 11

10 Le nombre d'heures de réservation par appareil et par laboratoire

```
SELECT ref_app, ref_lab,
       SUM(nb_heures) AS total_heures
FROM reservation
GROUP BY ref_app, ref_lab;
```

Un laboratoire peut réserver le même appareil sur plusieurs créneaux : **SUM** additionne ces heures.

Pour afficher les noms, joindre appareil et laboratoire et ajouter les noms au GROUP BY.

11 Les noms des appareils dont le tarif horaire est inférieur à 10

```
SELECT nom_app
FROM appareil
WHERE tarif_horaire < 10;
```

Un nombre s'écrit sans apostrophes. Le signe < exclut la valeur 10 elle-même.

EXERCICE 3

Gestion de la fréquentation des cinémas

Des cinémas diffusent des films, semaine après semaine.

EXERCICE 3 · MODÈLES

Le modèle logique

```
cinema(ref_cine, nom_cine, ville)
film(ref_film, titre, annee, tarif)
semaine(ref_semaine, date_debut)
diffusion(#ref_cine, #ref_film, #ref_semaine, entrees)
```

Souligné : clé primaire · # : clé étrangère

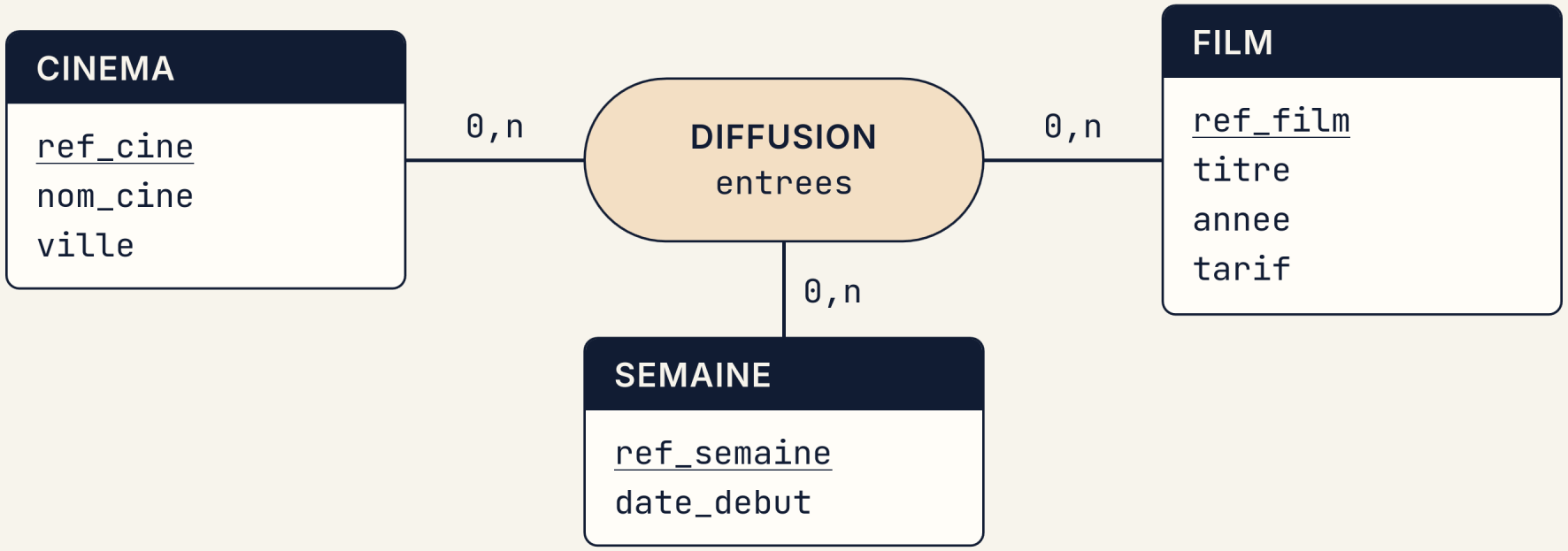
Chaque INSERT du sujet donne une table et la liste de ses colonnes.

diffusion ne contient que les références des trois autres tables, plus entrees : c'est une table d'association.

Sa clé primaire réunit les trois références, qui sont aussi trois clés étrangères.

EXERCICE 3 · MODÈLES

Le modèle conceptuel



Association ternaire : une diffusion relie un cinéma, un film et une semaine, et porte les entrées.

Cardinalités 0,n : le sujet n'impose aucun minimum, et la question 2 suppose des cinémas sans diffusion.

EXERCICE 3 · CRÉATION DES TABLES

Les tables sans clé étrangère

```sql
CREATE TABLE cinema (
    ref_cine  INTEGER PRIMARY KEY,
    nom_cine  VARCHAR(50),
    ville     VARCHAR(50)
);
```

```sql
CREATE TABLE semaine (
    ref_semaine  INTEGER PRIMARY KEY,
    date_debut   DATE
);
```

```sql
CREATE TABLE film (
    ref_film  INTEGER PRIMARY KEY,
    titre     VARCHAR(100),
    annee     INTEGER,
    tarif     NUMERIC(4,2)
);
```

NUMERIC(4,2) pour tarif : 9.50 a deux décimales.

DATE pour date_debut : 2026-10-05 est une date.

INTEGER pour annee, qui vaut 2001.

EXERCICE 3 · CRÉATION DES TABLES

La table d’association

```
CREATE TABLE diffusion (
    ref_cine     INTEGER,
    ref_film     INTEGER,
    ref_semaine  INTEGER,
    entrees      INTEGER,
    PRIMARY KEY (ref_cine, ref_film, ref_semaine),
    FOREIGN KEY (ref_cine)    REFERENCES cinema(ref_cine),
    FOREIGN KEY (ref_film)    REFERENCES film(ref_film),
    FOREIGN KEY (ref_semaine) REFERENCES semaine(ref_semaine)
);
```

Clé primaire : les trois références réunies, sur une seule ligne.

Clés étrangères : une par table reliée, vers sa clé primaire.

Cette table se crée en dernier ; les INSERT suivent le même ordre.

EXERCICE 3 · REQUÊTES

Questions 1 et 2

1 Le contenu de la table cinema

```sql
SELECT * FROM cinema;
```

2 Le nom des cinémas n'ayant enregistré aucune entrée

AVEC NOT IN

```sql
SELECT nom_cine
FROM cinema
WHERE ref_cine NOT IN (
    SELECT ref_cine FROM diffusion
);
```

AVEC LEFT JOIN

```sql
SELECT c.nom_cine
FROM cinema c
LEFT JOIN diffusion d
       ON d.ref_cine = c.ref_cine
WHERE d.ref_cine IS NULL;
```

Aucune entrée : le cinéma n'apparaît dans aucune ligne de diffusion. Les deux écritures donnent le même résultat.

EXERCICE 3 · REQUÊTES

Questions 3 à 5

3

Le titre des films, sans doublons

```
SELECT DISTINCT titre
FROM film;
```

Deux films différents peuvent porter le même titre : DISTINCT ne le cite qu'une fois.

4

Le nom des villes dans lesquelles un cinéma est implanté

```
SELECT DISTINCT ville
FROM cinema;
```

Une ville qui compte plusieurs cinémas n'est citée qu'une fois, grâce à DISTINCT.

5

Le titre et le tarif de chaque film

```
SELECT titre, tarif
FROM film;
```

Deux colonnes, séparées par une virgule ; toutes les lignes sont gardées.

EXERCICE 3 · REQUÊTES

Question 6

6 Le titre et les entrées de chaque film diffusé au cinéma « Rex »

```sql
SELECT f.titre, d.entrees
FROM diffusion d
JOIN film f   ON f.ref_film = d.ref_film
JOIN cinema c ON c.ref_cine = d.ref_cine
WHERE c.nom_cine = 'Rex';
```

titre est dans film, **entrees** dans diffusion, et la condition porte sur cinema.

Pas de DISTINCT : chaque semaine de diffusion garde sa ligne et ses entrées.

La table semaine ne sert pas ici : inutile de la joindre.

EXERCICE 3 · REQUÊTES

Question 7

7 La recette de chaque diffusion

```
SELECT d.ref_cine, d.ref_film, d.ref_semaine,
       d.entrees * f.tarif AS recette
FROM diffusion d
JOIN film f ON f.ref_film = d.ref_film;
```

Recette d'une diffusion = entrées × tarif du film.

entrees est dans diffusion, **tarif** dans film : la jointure est nécessaire.

Les trois références identifient la diffusion ; **AS** nomme la colonne calculée.

EXERCICE 3 · REQUÊTES

Question 8

8 Les films projetés par le cinéma « Rex » la semaine du 2026-10-05

```sql
SELECT DISTINCT f.titre
FROM diffusion d
JOIN film f    ON f.ref_film    = d.ref_film
JOIN cinema c  ON c.ref_cine    = d.ref_cine
JOIN semaine s ON s.ref_semaine = d.ref_semaine
WHERE c.nom_cine = 'Rex'
  AND s.date_debut = '2026-10-05';
```

Trois tables utiles autour de diffusion : film pour l'affichage, cinema et semaine pour les conditions.

La date s'écrit entre apostrophes, au format année-mois-jour.

DISTINCT par prudence : deux cinémas de villes différentes peuvent s'appeler Rex.

EXERCICE 3 · REQUÊTES

Question 9

9 La liste des diffusions de films dont le titre contient « am »

```
SELECT d.*
FROM diffusion d
JOIN film f ON f.ref_film = d.ref_film
WHERE f.titre ILIKE '%am%';
```

d.* désigne toutes les colonnes de diffusion, et elles seules.

ILIKE ignore la casse. Avec LIKE, « Amelie » ne serait pas trouvé, à cause de son A majuscule.

Autre écriture, valable hors de PostgreSQL :

```
WHERE LOWER(f.titre) LIKE '%am%'
```

EXERCICE 3 · REQUÊTES

Questions 10 et 11

10 Les entrées totales par film et par cinéma

```
SELECT ref_film, ref_cine,
       SUM(entrees) AS total_entrees
FROM diffusion
GROUP BY ref_film, ref_cine;
```

Un cinéma peut diffuser le même film plusieurs semaines : **SUM** additionne ces entrées.

Pour afficher les titres et les noms, joindre film et cinema et les ajouter au GROUP BY.

11 Les titres des films dont le tarif est inférieur à 10

```
SELECT titre
FROM film
WHERE tarif < 10;
```

Un nombre s'écrit sans apostrophes. Le tarif 9.50 convient ; 10 serait exclu.